

MRMI Gateway

Architecture Decision Record

Multi-Regional Multi-App Interlock

v0.9 — Universal Federation Protocol

ADR ID	MRMI-ADR-001
Status	PROPOSED
Author	Božidar Tadić
Version	0.9 — Universal Federation Protocol Layer: Schema Registry (ADR-015), Protocol Specification, Domain Adapters, Financial Corridors
Supersedes	v0.8
Covers App	v0.1 (target) · v0.2–v1.0 (roadmap)
Created	2025

"Legal compliance is not a deployment concern — it is an architectural constraint enforced at the transport layer."

"A universal federation protocol does not own the data it carries — it only guarantees that data reaches the right jurisdiction, intact."

0. Changelog

v0.8 → v0.9

Schema Layer (ADR-015) ★	Formal Schema Registry introduced. Gateway becomes payload-agnostic by design. Domain adapters: messaging (default), iso20022, hl7fhir, edifact, custom. Gateway validates envelope only; payload semantics belong to the domain.
Protocol Specification (§1.3) ★	MRMI formally repositioned as a protocol, not only a gateway implementation. Envelope contract is implementation-independent. Any language may implement a compatible node without using the Go reference implementation.

Financial Corridors (ADR-016) ★	ISO 20022 domain adapter with financial-specific policy extensions: settlement finality, cutoff times, idempotency guarantees for payment messages.
Jurisdiction Policy extended (ADR-005)	jurisdiction_isolation policy added: payload type may restrict which node tiers and regions it transits, independently of the sender/recipient region policy.
Open Questions updated	Schema versioning governance, financial corridor compliance framing, opaque_token lifetime, discovery rate limiting added.
Risks updated	Schema adapter non-compliance risk and financial corridor regulatory risk added.
Roadmap updated	ADR-015 Schema Registry in v0.1 scope. Financial corridor (iso20022) in v0.2. Domain adapter SDK helpers in v0.3.

Rows marked ★ are new in v0.9. Green background = new or significantly expanded.

1. Context

Modern messaging applications operate across a fragmented global regulatory landscape. Frameworks such as Russia's 152-ФЗ, the EU's GDPR, and Kazakhstan's data localisation laws impose strict requirements on where user data may reside and how it may cross borders. Existing federated protocols (Matrix, XMPP) treat cross-border data flow as an implementation concern rather than an architectural guarantee.

MRMI Gateway is an open-source federation middleware layer — analogous to Apache Kafka for messaging infrastructure — that any application can integrate. It provides inter-region message routing, identity resolution, at-least-once delivery, policy enforcement, and verifiable audit trails, while remaining agnostic to the business logic of the application built on top of it.

"Core insight: Legal compliance is not a deployment concern. It is an architectural constraint that must be enforced at the transport layer — not bolted on afterwards by operators."

1.1 Target Use-Case — v0.1

Primary corridor: Messaging between applications in RU, BY, KZ, AM, RS — regions with similar data sovereignty concerns, existing commercial relationships, and no current cross-border protocol enforcement. EU/US corridors are deferred to v1.0.

1.2 Scope

v0.1 — In scope: Federation gateway core, regional node (Go), policy engine with compliance profiles, Merkle audit log + DNS TXT + HTTPS fallback, signed policy configs with `applicable_law`, identity resolution (T0–T3) + revocation (ADR-010), at-least-once delivery + DLQ + retry, horizontal scaling / HA strategy (ADR-011), .NET SDK v0.1, Go CLI client, Management API read-only (ADR-014), hosted node model, **Schema Registry v0.1 (ADR-015)**.

v0.2 — In scope: Federated user discovery (ADR-012), push notification webhook contract, app namespace isolation, Management API write endpoints, **ISO 20022 financial domain adapter (ADR-016)**.

v0.3 — In scope: Node dashboard UI, storage backends, cache layer, dynamic node discovery, **domain adapter SDK helpers**.

v1.0 — In scope: DHT routing, EU/US corridor, governance model, operator certification programme, **Schema Registry governance**.

Out of scope (all versions): Full messenger UI, media storage / CDN, end-user authentication / KYC, billing, blockchain consensus, full traffic anonymity.

1.3 Protocol vs. Implementation — NEW in v0.9

★ NEW in v0.9

Starting with v0.9, MRMI is formally repositioned as a **protocol specification**, not only a gateway implementation. The Go binary is the reference implementation. The protocol itself is implementation-independent.

This distinction matters for adoption: a Java shop, a Python ML pipeline, or a financial institution running a JVM stack can implement a fully compatible MRMI node without touching the Go codebase. The envelope contract (§4.1), the policy engine interface (ADR-005), and the audit log format (ADR-005) are the normative protocol surface.

The analogy is TCP/IP: the protocol is defined independently of any OS kernel implementation. MRMI follows the same principle. The Go reference implementation ships first; alternative implementations are explicitly welcome and will be tracked in the ecosystem registry.

2. Architecture Decision Records

ADR-001 through ADR-014 are carried forward from v0.8 without modification. This section documents the new decisions introduced in v0.9.

ADR-015 — Schema Registry ★ NEW

★ NEW in v0.9

Problem

MRMI Gateway is architecturally payload-agnostic: the envelope carries an opaque encrypted blob. However, without a formal mechanism for declaring payload types, every integration requires out-of-band agreement between the sending and receiving application. This limits adoption beyond the messaging domain.

As MRMI expands into financial messaging (ISO 20022), healthcare (HL7/FHIR), logistics (EDIFACT), and government document exchange, a structured mechanism is needed to register, version, and route domain-specific payload types — without compromising the gateway's payload opacity.

Decision

Introduce a **Schema Registry** as a first-class MRML component. The registry operates at the Schema Layer, sitting above the Transport Layer (routing, delivery) and below the Application Layer (business logic).

Three-Layer Architecture

MRMI Architecture – Three Layers (v0.9)

```

■ APPLICATION LAYER ■
■ Business logic, domain semantics, user-facing features ■
■ (Messaging app, Financial system, EHR, Logistics platform) ■
■ SCHEMA LAYER ← NEW in v0.9 ■
■ Schema Registry: domain adapter registration & validation ■
■ Adapters: messaging | iso20022 | hl7fhir | edifact | custom ■
■ Gateway validates envelope; adapter validates payload shape ■
■ TRANSPORT LAYER (unchanged from v0.8) ■
■ Routing · Delivery · Policy · Audit · mTLS · Revocation ■
■ Does NOT open payload – envelope only ■

```

Envelope — schema_type field

```
message Envelope {
  // ... existing fields (v0.8) ...
  string schema_type    = 12; // NEW: "messaging" | "iso20022" | "hl7fhir"
                                //      | "edifact" | "custom:<identifier>"
  string schema_version = 13; // NEW: semver – "1.0.0"
  bytes  payload        = 8;  // opaque, E2E encrypted – unchanged
}
```

The gateway reads `schema_type` only to: (1) apply `jurisdiction_isolation` policy (ADR-005 extension), (2) record `schema_type` in the audit log entry. It does **not** decode or validate the payload content — that remains the application's responsibility.

Built-in Domain Adapters — v0.1

schema_type	version	Description
messaging	1.0.0	Default. Chat messages, notifications. Existing behaviour unchanged.
iso20022	1.0.0	Financial messages. MT/MX format hint. Triggers financial policy extensions (ADR-016). v0.2.
hl7fhir	1.0.0	Healthcare. FHIR R4 resource envelope hint. Jurisdiction policy: strict only. v0.3.
edifact	1.0.0	Logistics. UN/EDIFACT. Supply chain corridors. v0.3.
custom:	any	Any domain. Operator-registered. No built-in policy extensions.

Schema Registry — Custom Registration

```
# TOML — operator registers a custom domain adapter
[schema_registry]
  [[schema_registry.adapters]]
    type           = "custom:gov-doc-exchange"
    version        = "1.0.0"
    description    = "Government document exchange — RS Ministry of Interior"
    jurisdiction_policy = "strict"    # force strict profile for this schema type
    allow_tiers    = ["regional"]    # only transit through regional nodes
    contact        = "ops@example.rs"

  [[schema_registry.adapters]]
    type           = "custom:energy-iec61968"
    version        = "2.1.0"
    description    = "IEC 61968/61970 — Energy grid management messages"
    jurisdiction_policy = "balanced"
    allow_tiers    = ["regional", "alliance"]
```

Audit Log — schema fields added

```
AuditEntry {
  // ... existing fields (v0.8) ...
  schema_type:  string  // NEW — e.g. "iso20022", "messaging"
  schema_version: string // NEW — e.g. "1.0.0"
}
```

Design Principle

"The gateway never opens the payload. schema_type is a routing and policy hint declared by the sender — not a content inspection mechanism. Payload validation is the application's responsibility."

ADR-005 Extension — jurisdiction_isolation Policy ★ NEW

★ NEW in v0.9

v0.8 policy engine controls routing based on sender/recipient region. v0.9 adds **jurisdiction_isolation**: a payload type may independently restrict which node tiers and jurisdictions it transits, regardless of sender/recipient regions.

Use case: a financial institution uses MRMI for both chat messages (messaging) and ISO 20022 payment messages (iso20022). Chat may transit global nodes; payment messages must only transit regional nodes in declared jurisdictions.

```
# TOML — jurisdiction_isolation per schema_type
[policy.jurisdiction_isolation]
  [[policy.jurisdiction_isolation.rules]]
```

```

schema_type      = "iso20022"
allow_tiers      = ["regional"]          # never via global or alliance nodes
allow_jurisdictions = ["RU", "RS", "BY"] # explicit whitelist
require_profile   = "strict"             # force strict regardless of node default

[[policy.jurisdiction_isolation.rules]]
schema_type      = "hl7fhir"
allow_tiers      = ["regional", "alliance"]
require_profile   = "strict"

[[policy.jurisdiction_isolation.rules]]
schema_type      = "messaging"
allow_tiers      = ["regional", "alliance", "global"] # default – unrestricted

```

ADR-016 — Financial Domain Adapter (ISO 20022) ★ NEW

★ NEW in v0.9

Context

Financial messaging systems (SWIFT MT/MX, ISO 20022) are structurally identical to general-purpose messaging at the transport level: structured message → routing → delivery → confirmation. MRMI's transport layer already provides the required primitives. ADR-016 defines the financial-specific policy extensions that make MRMI suitable for regulated financial message corridors.

The target use case is not replacement of SWIFT or CIPS. It is providing a decentralised, sovereignty-preserving federation layer that existing financial systems can adopt as a routing substrate — particularly for corridors where SWIFT access is restricted or commercially undesirable.

Financial-Specific Policy Extensions

Extension	Specification
Settlement Finality	Once a message with <code>schema_type=iso20022</code> reaches ACK from the destination node, it is marked final. No replay or cancellation via MRMI. Application-level reversal is a separate message.
Cutoff Times	Operators declare processing windows per corridor. Messages arriving outside the window are held in DLQ until the next window opens — not rejected. Configurable per <code>allow_to</code> region pair.
Idempotency TTL	iso20022 messages default to dedup TTL = 72h (strict profile) regardless of operator profile selection. Financial messages must not be duplicated even across multi-day settlement windows.
BIC/IBAN Routing Hint	Optional envelope field: <code>routing_hint</code> . For iso20022, this carries a BIC prefix (first 6 chars) as a routing hint — node uses it to select the optimal regional path. Never used for identity — only for path optimisation.

Audit Retention	iso20022 audit entries are flagged retain_long=true. Operators must configure Merkle log retention >= 7 years for financial corridors. Enforcement is operator responsibility; flag provides a machine-readable signal for log management tools.
-----------------	--

TOML — Financial Corridor Configuration

```
[schema_registry.adapters.iso20022]
enabled          = true
version          = "1.0.0"
dedup_ttl_h      = 72          # override – always 72h for financial messages
settlement_finality = true
retain_long      = true        # audit log retention signal

[schema_registry.adapters.iso20022.cutoff_windows]
[schema_registry.adapters.iso20022.cutoff_windows."RU-RS"]
open  = "08:00"
close = "17:00"
tz    = "Europe/Belgrade"

[schema_registry.adapters.iso20022.cutoff_windows."RU-BY"]
open  = "08:00"
close = "18:00"
tz    = "Europe/Minsk"

[schema_registry.adapters.iso20022.routing_hint]
enabled = true
field   = "bic_prefix"  # first 6 chars of BIC used for path selection
```

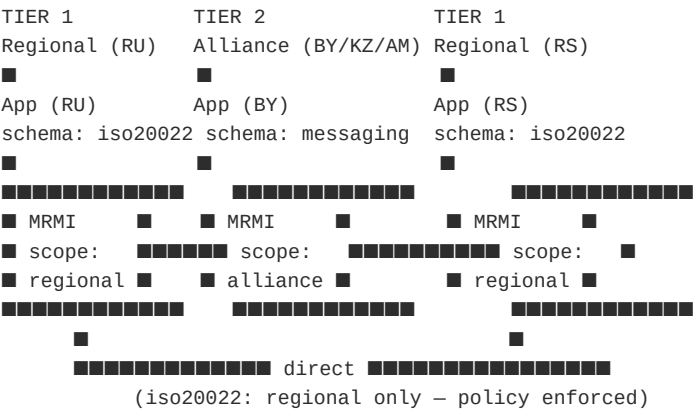
Compliance Note

■ ■ The iso20022 adapter provides technical infrastructure for financial message routing. It does not constitute a payment system, a banking licence, or a money transmission service. Operators deploying iso20022 corridors must obtain all applicable regulatory approvals under their jurisdiction. MRMI is a routing layer — legal accountability for financial compliance rests entirely with the operator.

■ ■ Operators must verify with legal counsel that MRMI financial corridors do not constitute sanctions circumvention under applicable law. The decentralised architecture does not provide legal immunity — it provides technical sovereignty. These are distinct.

3. Architecture Overview

MRMI GATEWAY – MESSAGE FLOW (v0.9)



Schema Layer (v0.9):

- Each node: validate schema_type in envelope header
- Apply jurisdiction_isolation policy for this schema_type
- Record schema_type + schema_version in audit entry
- Forward opaque payload – never inspect content

Transport Layer (unchanged from v0.8):

1. Validate idempotency key (dedup TTL per profile / schema override)
2. Check CRL (revocation)
3. Policy engine + profile + routing tier + jurisdiction_isolation
4. Apply padding + jitter
5. Sign + forward envelope (gRPC/mTLS)
6. Write Merkle audit entry (includes schema_type, node_scope)

4. Data & Metadata Privacy

Sections 4.1 (Envelope Structure), 4.2 (Metadata Minimisation), and 4.3 (Data Transit Policy) are carried forward from v0.8 without modification. The envelope structure gains two new fields in v0.9: `schema_type` (field 12) and `schema_version` (field 13). These are transport-layer hints only — they do not expose payload content.

The gateway's opacity guarantee is unchanged: the transport layer never inspects, decrypts, or validates the payload field (field 8). `schema_type` is declared by the sender in the envelope header and trusted at the same level as `sender_region` — operator policy may restrict which schema types are permitted on a given node.

6. Risks & Mitigations (v0.9 additions)

Risks from v0.8 are carried forward. New risks introduced in v0.9:

Risk	Severity	Mitigation
Schema adapter non-compliance	■ MED	An operator declares <code>schema_type=iso20022</code> but sends non-financial payload. Gateway cannot verify — opacity is by design. Mitigation: <code>jurisdiction_isolation</code> policy restricts which nodes may carry <code>iso20022</code> traffic; mTLS + signed policy config identifies the declaring operator. Misuse is auditable post-fact.
Financial corridor regulatory risk	■ HIGH	<code>iso20022</code> corridors may be classified as payment infrastructure by some regulators, requiring licences not required for messaging. Mitigation: compliance note in ADR-016 explicitly disclaims payment system status. Operators must conduct their own legal analysis. Gateway documentation makes this boundary explicit.
Schema Registry governance fragmentation	■ MED	Without a neutral governance body, <code>schema_type</code> values may fragment (different operators using incompatible custom: identifiers for the same domain). Mitigation: <code>schema_type</code> registry maintained as a community document in the GitHub repo. Formal governance deferred to v1.0 foundation model.
Protocol ossification via schema lock-in	■ LOW	Early <code>schema_type</code> values becoming de-facto standards before the protocol is mature. Mitigation: <code>schema_version</code> field allows non-breaking evolution. Breaking changes require <code>schema_type</code> version bump, not a new type identifier.

8. Open Questions (v0.9 additions)

Existing open questions from v0.8 are carried forward. New questions for community discussion:

Schema Registry governance	Who maintains the registry of valid <code>schema_type</code> values in the decentralised model? GitHub repo (author-controlled) for v0.1–v0.3. Foundation/committee for v1.0. Transition mechanism?
Financial corridor compliance framing	Under which jurisdictions does an iso20022 corridor require a payment institution licence vs. qualifying as 'technical routing infrastructure'? Legal analysis needed per corridor before v0.2 financial adapter launch.
Custom <code>schema_type</code> namespace	Should custom: identifiers be namespaced (<code>custom:org.example.gov-docs</code>) to prevent collision? Or is a flat registry sufficient for the expected number of adapters?
Schema validation at node boundary	Should nodes optionally validate envelope <code>schema_type</code> against a registered format schema (JSON Schema / Protobuf descriptor), or remain strictly opaque? Opt-in validation would help operators detect misconfigured senders without compromising the opacity guarantee for compliant traffic.
iso20022 <code>cutoff</code> <code>window</code> enforcement	Cutoff times in ADR-016 are declared per corridor. If the destination node has a different cutoff window than the origin node, which takes precedence? Proposed: destination node's window is authoritative.

12. Milestones — v0.9 Additions

v0.1 — Additional Milestones (Schema Registry)

#	Task	Owner
13	ADR-015: Schema Registry TOML config + schema_type/schema_version envelope fields (proto update)	Božidar Tadić
14	Gateway: jurisdiction_isolation policy engine extension — per-schema-type tier and jurisdiction restrictions	Božidar Tadić
15	Audit log: schema_type + schema_version fields in AuditEntry (proto + Merkle log update)	Božidar Tadić
16	messaging adapter: default adapter, backward-compatible with all v0.8 envelopes	Božidar Tadić
17	.NET SDK v0.1: SchemaType enum + schema_version in MrmiClient send API	Božidar Tadić
18	Protocol Specification document: envelope contract, policy interface, audit format — implementation-independent	Božidar Tadić

v0.2 — Additional Milestones (Financial Adapter)

#	Task	Owner
11	ADR-016: iso20022 domain adapter — settlement finality, cutoff windows, BIC routing hint	Božidar Tadić
12	iso20022 policy extensions: dedup_ttl_h override, retain_long flag, cutoff window enforcement	Božidar Tadić
13	Legal analysis: iso20022 corridor compliance framing per target jurisdiction (RU, RS, BY)	Božidar Tadić + counsel
14	Python SDK v0.2: schema_type support, iso20022 helper (envelope construction)	Open for contributors

v0.3 — Additional Milestones (Domain Adapter Infrastructure)

#	Task	Owner
7	hl7fhir adapter: HL7 FHIR R4 schema hint, strict-only jurisdiction policy	Open for contributors
8	edifact adapter: UN/EDIFACT schema hint, logistics corridor support	Open for contributors
9	SDK helpers: domain-specific envelope builders for each built-in adapter (Go + .NET)	Božidar Tadić
10	Schema Registry UI: list registered adapters, per-adapter policy view, custom adapter registration	Open for contributors

13. Roadmap (updated v0.9)

Version	Focus	Key Deliverables
v0.1	Core protocol + Schema Registry	Routing, policy, audit, delivery, mTLS, revocation, node tiers, hosted nodes, read-only API, seed corridor RS↔RU, Schema Registry (ADR-015), jurisdiction_isolation policy, messaging adapter, Protocol Specification document
v0.2	User discovery + Financial adapter	Federated lookup, opaque token, app namespace isolation, auto-accept, push webhook, write API, ISO 20022 financial domain adapter (ADR-016), cutoff windows, settlement finality
v0.3	Operator infrastructure + Domain adapters	Storage backends, cache layer, dynamic node discovery, dashboard UI, HL7/FHIR adapter, EDIFACT adapter, domain adapter SDK helpers, alliance/global node operator guide
v1.0	Scale + Governance	DHT routing, EU alliance node, RU↔EU via RS neutral relay, governance model, operator certification, reputation system v1.0, Schema Registry governance (foundation/committee)
post-v1.0	Extensions + New Domains	Video calls, WebRTC signaling extension, additional SDK languages, energy sector adapter (IEC 61968/61970), government document exchange adapter

Green row = v0.1 — current active milestone. Principle: each version is independently deployable and useful.

Appendix C — Schema Registry TOML Reference ★ NEW

★ NEW in v0.9

Complete reference configuration for the Schema Registry. This section replaces operator-specific out-of-band agreements for payload type declaration.

C.1 Minimal — messaging only (default, backward-compatible)

No [schema_registry] section required for messaging-only deployments.
All envelopes without schema_type field default to schema_type="messaging".

C.2 Financial corridor — iso20022 enabled

```
[schema_registry]
  [[schema_registry.adapters]]
    type      = "iso20022"
    version   = "1.0.0"
    enabled   = true

    [schema_registry.adapters.iso20022]
      dedup_ttl_h      = 72
      settlement_finality = true
      retain_long      = true

    [schema_registry.adapters.iso20022.cutoff_windows."RU-RS"]
      open  = "08:00"
      close = "17:00"
      tz    = "Europe/Belgrade"

[policy.jurisdiction_isolation]
  [[policy.jurisdiction_isolation.rules]]
    schema_type      = "iso20022"
    allow_tiers      = ["regional"]
    allow_jurisdictions = ["RU", "RS", "BY", "KZ", "AM"]
    require_profile   = "strict"
```

C.3 Multi-domain — messaging + financial + custom

```
[schema_registry]
  [[schema_registry.adapters]]
    type      = "messaging"
    version   = "1.0.0"
    enabled   = true

    [[schema_registry.adapters]]
      type      = "iso20022"
      version   = "1.0.0"
      enabled   = true

    [[schema_registry.adapters]]
      type      = "custom:gov-rs-doc-exchange"
      version   = "1.0.0"
      enabled   = true
      description = "RS government document exchange"
      jurisdiction_policy = "strict"
      allow_tiers      = ["regional"]
      contact          = "ops@gov.rs"

[policy.jurisdiction_isolation]
  [[policy.jurisdiction_isolation.rules]]
    schema_type      = "iso20022"
    allow_tiers      = ["regional"]
    allow_jurisdictions = ["RU", "RS"]
    require_profile   = "strict"
```

```
[[policy.jurisdiction_isolation.rules]]
schema_type      = "custom:gov-rs-doc-exchange"
allow_tiers      = ["regional"]
allow_jurisdictions = ["RS"]
require_profile   = "strict"
```

This ADR is a living document. All decisions marked PROPOSED are open for community discussion.

MRMI Gateway Project · Open Source · v0.9 · 2025 · Božidar Tadić